

Contents

Hướng dẫn đọc và tìm code — PM QLKT

1. Nắm tổng quan trước khi mở file

2. Kiến trúc 2 tầng

3. 4 vai trò — biết role nào làm gì ở đâu

4. Đi theo 1 feature (golden path)

5. 7 loại khen thưởng — biết code ở đâu

6. Chuỗi danh hiệu (BKBQP/CSTDTQ/BKTTCP) — phần đắt giá khi bảo vệ

7. Cheatsheet — tìm code nhanh

8. Naming conventions giúp tìm nhanh

9. Pitfalls — chỗ dễ nhầm

10. Khi muốn thêm 1 loại khen thưởng mới (quy trình)

11. Lệnh hữu ích khi đọc/debug code

12. Đọc test để hiểu business rule (tip cuối)

1

1

2

2

2

3

3

4

4

4

5

5

Hướng dẫn đọc và tìm code — PM QLKT

Mục tiêu: giúp người đọc mới (bạn cùng nhóm, thầy phản biện, người tiếp nhận sau bảo vệ) nắm được cấu trúc + biết tìm code ở đâu trong 30 phút.

1. Nắm tổng quan trước khi mở file

Đọc theo thứ tự này, mỗi mục 5-10 phút là đủ:

1. README.md (nếu có) hoặc report/BAO_CAO.md §4.1 — kiến trúc tổng thể

2. CLAUDE.md (root) — convention + Quick Commands + Architecture

3. BE-QLKT/CLAUDE.md — cấu trúc thư mục BE, anti-patterns

4. FE-QLKT/CLAUDE.md — cấu trúc thư mục FE

5. docs/diagrams/03-architecture.md — sơ đồ kiến trúc layered

6. docs/diagrams/06-erd.md — schema DB

Sau bước này bạn đã có “bản đồ”. Chưa cần đọc code.

2. Kiến trúc 2 tầng

FE-QLKT (Next.js 14 App Router)

src/app/<role>/<feature>/page.tsx

↓ (gọi apiClient)

src/lib/api/<domain>.ts

↓ (axios)

src/lib/axiosInstance.ts

↓ HTTP

BE-QLKT (Express + Prisma)

src/routes/<feature>.route.ts

↓ (middleware chain)

src/middlewares/{auth,validate,auditLog}.ts

↓

src/controllers/<feature>.controller.ts

↓

src/services/<feature>.service.ts

↓

src/repositories/<entity>.repository.ts

↓

src/models/index.ts (Prisma)

↓

PostgreSQL

Quy tắc vàng: - Controller KHÔNG import Prisma trực tiếp — luôn qua repository hoặc service - Service chứa business logic; Repository chỉ wrap Prisma - FE luôn gọi qua apiClient từ @/lib/api, không fetch trực tiếp

3. 4 vai trò — biết role nào làm gì ở đâu

SUPER_ADMIN > ADMIN > MANAGER > USER

Tim code theo role: mỗi role có thư mục riêng trong FE.

Role	FE folder	Quyền chính
SUPER_ADMIN	FE-QLKT/src/app/super-admin/	Quản lý danh mục đơn vị, dev zone, backup
ADMIN	FE-QLKT/src/app/admin/	Duyệt đề xuất, quản lý khen thưởng, nhập Excel
MANAGER	FE-QLKT/src/app/manager/	Tạo đề xuất, xem khen thưởng đơn vị mình
USER	FE-QLKT/src/app/user/	Xem hồ sơ cá nhân, nhận thông báo

BE phân quyền qua middleware `requireRole` trong route file:

```
router.post('/', verifyToken, requireAdmin, ...)
```

4. Đi theo 1 feature (golden path)

Ví dụ: “Khi MANAGER tạo đề xuất khen thưởng cá nhân hằng năm, code chạy thế nào?”

1. **Trigger UI:** FE-QLKT/src/app/manager/proposals/create/page.tsx — form submit
2. **API call:** `apiClient.submitProposal()` trong FE-QLKT/src/lib/api/proposals.ts
3. **HTTP** → POST /api/proposals
4. **Route:** BE-QLKT/src/routes/proposal.route.ts — middleware chain `verifyToken` → `requireManager` → `validate(schema)`
5. **Validation:** BE-QLKT/src/validations/proposal.validation.ts — Zod schema
6. **Controller:** BE-QLKT/src/controllers/proposal.controller.ts — gọi service
7. **Service:** BE-QLKT/src/services/proposal/submit.ts — orchestration
8. **Strategy dispatch:** BE-QLKT/src/services/proposal/strategies/index.ts — REGISTRY chọn `caNhanHangNamStrategy.ts` theo `loai_de_xuat`
9. **Eligibility check:** BE-QLKT/src/services/eligibility/chainEligibility.ts cho rule chuỗi danh hiệu
10. **Repository:** BE-QLKT/src/repositories/proposal.repository.ts — INSERT vào DB
11. **Notification:** BE-QLKT/src/helpers/notification/awards.ts — gửi thông báo realtime qua Socket.IO
12. **Response** → FE update bảng

Đề hiểu 1 feature: đi đủ 12 bước trên là nắm được.

5. 7 loại khen thưởng — biết code ở đâu

Đây là phần phức tạp nhất (strategy pattern). Mỗi loại có:

Mã	Tên	Strategy file	Service file BE
CA_NHAN_HANG_NAM	Khen thưởng cá nhân hằng năm	proposal/strategies/caNhanHangNamStrategy.ts	services/caNhanHangNamAward.ts
DON_VI_HANG_NAM	Khen thưởng đơn vị hằng năm	proposal/strategies/donViHangNamStrategy.ts	services/donViHangNamAward.ts

Mã	Tên	Strategy file	Service file BE
NIEN_HAN	Huân chương niên hạn	proposal/strategies/tenureMedal/* (tenure-medals)	services/tenureMedal.ts
CONG_HIEN	Huân chương công hiến	proposal/strategies/longServiceMedal.ts	services/longServiceMedal.ts
KNC_VSNXD_QDNDVN	Huy chương kỷ niệm	proposal/strategies/commemorativeMedal.ts	services/commemorativeMedal.ts
HC_QKQT	Huân chương Quân kỳ quyết thắng	proposal/strategies/militaryFlag.service.ts	services/militaryFlag.service.ts
NCKH	Thành tích NCKH	proposal/strategies/scientificAchievement.ts	services/scientificAchievement.ts

Dispatch: `proposal/strategies/index.ts` map `loai_de_xuat` → `strategy`. Không có `if/else` dài.

6. Chuỗi danh hiệu (BKBQP/CSTDTQ/BKTTCP) — phần đắt giá khi bảo vệ

Đây là logic phức tạp nhất + được test kỹ nhất:

- **Config-driven:** `BE-QLKT/src/services/eligibility/chainAwardConfig.ts` chứa `PERSONAL_CHAIN_AWARDS + UNIT_CHAIN_AWARDS`
- **Core logic:** `BE-QLKT/src/services/eligibility/chainEligibility.ts` (hàm `checkChainEligibility`)
- **Context helper:** `BE-QLKT/src/services/profile/annual.ts` — `computeChainContext` tính chuỗi liên tục, cửa sổ trượt, etc
- **Tests:** `BE-QLKT/tests/services/eligibility-{bkbqp,cstdtq,bkttcp}-{personal,unit}.test.ts`

Quy tắc chu kỳ — đọc `CLAUDE.md` §Architecture mục “Chain cycle semantics” để hiểu BKBQP 2y, CSTDTQ 3y, BKTTCP 7y, lifetime block, cửa sổ trượt.

7. Cheatsheet — tìm code nhanh

Cần tìm...	Lệnh / Đường dẫn
Một API endpoint	<code>grep -rn "POST '/api/<path>'"</code> <code>BE-QLKT/src/routes/</code>
Component theo tên	<code>find FE-QLKT/src/components -name "<Name>*.tsx"</code>
Hằng số (vd: <code>DANH_HIEU_MAP</code>)	<code>BE-QLKT/src/constants/</code> hoặc <code>FE-QLKT/src/constants/</code>
Schema validation 1 endpoint	<code>BE-QLKT/src/validations/<entity>.validation.ts</code>
Audit log description builder	<code>BE-QLKT/src/helpers/auditLog/<domain>.ts</code>
Notification message builder	<code>BE-QLKT/src/helpers/notification/<domain>.ts</code>
Strategy của 1 loại đề xuất	<code>BE-QLKT/src/services/proposal/strategies/<type>Strategy.ts</code>
Eligibility rule chuỗi danh hiệu	<code>BE-QLKT/src/services/eligibility/chainEligibility.ts</code>
Test 1 chức năng	<code>BE-QLKT/tests/services/</code> hoặc <code>tests/approve/</code> hoặc <code>tests/scenarios/</code>
Format ngày	<code>FE-QLKT/src/lib/utils.ts</code> — <code>formatDate</code> , <code>formatDateTime</code>
API client method	<code>FE-QLKT/src/lib/api/<domain>.ts</code> (re-export trong <code>index.ts</code>)
Helper chuỗi danh hiệu FE	<code>FE-QLKT/src/lib/award/</code>

Cần tìm...	Lệnh / Đường dẫn
ERD diagram	BE-QLKT/prisma/ERD.svg hoặc docs/diagrams/06-erd.md

8. Naming conventions giúp tìm nhanh

Loại file	Pattern	Ví dụ
BE Route	<entity>.route.ts	account.route.ts
BE Controller	<entity>.controller.ts	account.controller.ts
BE Service	<entity>.service.ts hoặc <entity>/*.ts	account.service.ts, proposal/submit.ts
BE Repository	<entity>.repository.ts	account.repository.ts
BE Validation	<entity>.validation.ts	account.validation.ts
BE Constant	<entity>.constants.ts	roles.constants.ts
FE Component	PascalCase.tsx	PersonnelTable.tsx
FE Hook	use<X>.ts	useFetch.ts, useAuthGuard.ts
FE Page	app/<role>/<feature>/page.tsx	app/admin/awards/page.tsx
FE API module	lib/api/<domain>.ts	lib/api/proposals.ts
shadcn/ui	kebab-case.tsx	button.tsx (ngoại lệ — không sửa)

Biết pattern → đoán được tên file → không cần grep.

9. Pitfalls — chỗ dễ nhầm

- **co_quan_don_vi_id vs don_vi_truc_thuoc_id**: ưu tiên DVTT trước, CQDV sau. CQDV có thể là đơn vị cha. Xem CLAUDE.md §Architecture “Unit priority”.
- **data_* JSON columns**: 4 cột data_danh_hieu/thanh_tich/nien_han/cong_hien lưu JSON tự do, validate ở Zod + Strategy.
- **isLifetime: true**: BKTTCPC cá nhân chỉ nhận 1 lần — sau khi nhận thì BE trả gọi_y = "Phần mềm chưa hỗ trợ khen thưởng cao hơn...". Xem CLAUDE.md §Chain cycle semantics.
- **Cửa sổ trượt 3y/7y**: CSTDTQ check BKBQP trong 3 năm cuối từ year-1. BKTTCPC cá nhân check strict ==, unit BKTTCPC check >=.
- **Hierarchy of awards**: HCCSVV/HCBVTQ phải có rank dưới trước rồi mới được rank trên (Ba → Nhì → Nhất).
- **Test khi sửa eligibility**: bất kỳ thay đổi rule chuỗi nào → chạy BE-QLKT/tests/services/eligibility-*.test.ts (870+ ca).

10. Khi muốn thêm 1 loại khen thưởng mới (quy trình)

Đây là câu hỏi rất hay khi bảo vệ — “Hệ thống có dễ mở rộng không?”:

1. Thêm constant <type> vào BE-QLKT/src/constants/proposalTypes.constants.ts
2. Thêm schema Zod vào BE-QLKT/src/validations/proposal.validation.ts
3. Tạo Strategy file BE-QLKT/src/services/proposal/strategies/<type>Strategy.ts implement ProposalStrategy interface
4. Đăng ký vào REGISTRY ở BE-QLKT/src/services/proposal/strategies/index.ts
5. Thêm migration Prisma nếu cần bảng riêng (vd: NCKH có bảng ThanhTichKhoaHoc)
6. Viết test ở BE-QLKT/tests/services/eligibility-<type>.test.ts

7. FE: thêm option vào FE-QLKT/src/constants/proposal.constants.ts + component Step2SelectPersonnel + Step3SetTitles<Type>.tsx

KHÔNG cần sửa: route (proposal route generic), controller (dispatch qua REGISTRY), audit log helper (generic), notification helper (generic).

11. Lệnh hữu ích khi đọc/debug code

```
# BE - dev với watch mode
cd BE-QLKT && npm run dev

# BE - chạy 1 test cụ thể
cd BE-QLKT && npx jest tests/services/eligibility-bkbqp-personal.test.ts

# BE - chạy tất cả test
cd BE-QLKT && npx jest

# BE - xem schema DB qua GUI
cd BE-QLKT && npx prisma studio

# FE - dev
cd FE-QLKT && npm run dev

# FE - typecheck nhanh không build
cd FE-QLKT && npm run typecheck

# Tìm tất cả nơi gọi 1 hàm
grep -rn "functionName" BE-QLKT/src/ FE-QLKT/src/

# Tìm hằng số được dùng ở đâu
grep -rn "PROPOSAL_TYPES.NIEN_HAN" BE-QLKT/src/ FE-QLKT/src/
```

12. Đọc test để hiểu business rule (tip cuối)

Test thường rõ ràng hơn doc. Khi không hiểu một rule:

1. Tìm test có tên gần nhất: BE-QLKT/tests/services/eligibility-*.test.ts
2. Đọc describe/it block name → hiểu rule đang kiểm tra gì
3. Đọc errorMessagees.ts trong cùng folder → các message đã chốt

Vd để hiểu rule BKBQP cá nhân: mở tests/services/eligibility-bkbqp-personal.test.ts, đọc 20-30 it () đầu tiên là nắm được toàn bộ rule.